

Login authentication options

User

Log in fix. We should be able to log in with password or supabase auth. add one button like over the google one for supabase. I think we are using supabase auth all the time, is it?

Assistant

Yes — checked all the imports (`authContext.tsx`, `signUp.tsx`, `moviesContext.tsx`) and everything goes through `supabase-api.ts`, which calls real Supabase Auth methods. The existing sign-in form already logs in through Supabase using the password grant. `auth-api.ts` (the `localStorage` version) isn't imported anywhere — it's dead code from before the migration.

So there's no “password vs Supabase” split to fix — there's one Supabase auth method (password) and a fake Google button (`alert(...)`). The tutorial wires that Google button to real Supabase OAuth as a second working method:

1. Enable Google provider in Supabase dashboard + Google Cloud OAuth client (config only, no code).
2. Add `loginWithGoogle` to `supabase-api.ts`.
3. Expose it through `authContext.tsx`.
4. Wire the button in `signIn.tsx`.
5. Test end to end.

Each code step shows a diff against the current file to make the edits.

User

you got it wrong. maybe my mistake. I need to implement log in with password. The password which is saved when signing up. or Supabase auth. for supabase auth, I need a button like the google/facebook ones .

Assistant

Got it — that's a different setup than what was built. Let me make sure the right thing gets built before writing the tutorial.

User

Continue from where you left off.

Assistant

No response requested.

User

maybe I am wrong. where is the password being saved? in the supabase? Is it hidden there?

Assistant

Yes — hidden there. When `signUp.tsx` calls `supabase.auth.signUp({ email, password })` (via `supabase-api.ts`), Supabase's Auth service (GoTrue) hashes the password with `bcrypt` and stores it in the `auth.users` table, in a column called `encrypted_password`, inside the project's own Postgres database.

Two things worth knowing about that table:

- It lives in the `auth` schema, separate from `public`, and isn't exposed through the API — the anon key can't `SELECT` from it like a normal table, even with RLS off. The only way to check a password is via `signInWithPassword`, which runs server-side and returns a session or an error, never the hash itself.
- It's a proper hash, not reversible — nobody (not even the project owner) can look up “what was the plaintext password.” Login only works by re-hashing the attempt and comparing.

So this resolves the earlier confusion: when signing up through `supabase-api.ts` today, the password is already saved — in Supabase's database, not lost anywhere. There's no separate “local” password store unless deliberately added (that's what the unused `auth-api.ts` + `localStorage` would be, a second, independent copy of the password on the user's own browser).

Given that, is the goal still two separate login paths (local password check via `auth-api.ts` + a distinct “Sign in with Supabase” button), or is the existing single Sign In button (already backed by Supabase) enough — with the only real addition being the Google button?

User

ok No problem them

Assistant

Good — that keeps things simple: one Sign In button (already Supabase-backed, no changes needed) plus a working Google button.

That's exactly what the tutorial already built — `google-oauth-login-tutorial.md` in the project folder. It's still accurate: enable Google in the Supabase dashboard, add `loginWithGoogle` to `supabase-api.ts`, expose it through `authContext.tsx`, and wire the existing (currently fake) Google button in `signIn.tsx`. No changes needed based on this conversation.